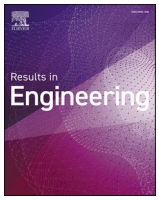
Results in Engineering 26 (2025) 104661



Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering



Research paper



An energy-efficient obstacle-crossing control framework for quadruped robots

Jiang Han, Baishu Wan, Yilin Zheng, Zhigong Song *

Jiangsu Provincial Key Laboratory of Food Advanced Manufacturing Equipment Technology, School of Mechanical Engineering, Jiangnan University, Wuxi 214122, China

ARTICLE INFO

Keywords:
Legged robots
Embodied intelligence
Physical laws
Energy-efficient obstacle crossing

ABSTRACT

The ability of a quadruped robot to cross obstacles is a crucial metric for assessing its adaptability in complex environments. Traditional control methods depend on precise physical modeling, which struggles to adapt to complex environments. Nowadays, embodied intelligence has become an important concept for describing agent as learning through environmental interactions. In recent years, techniques like deep reinforcement learning and imitation learning, designed to address interaction challenges, have achieved significant success in robot control. However, many challenges remain, including complex reward mechanism design, poor model generalization, and insufficient expression of physical laws. To this end, a novel energy-efficient obstacle-crossing control framework is developed, which combines the data-driven method of adversarial motion prior and the energy consumption knowledge of physics. This allows the quadruped robot to generate multiple feasible and lowest energy consumption trajectories according to the obstacle information and its current state, enabling it to successfully complete the obstacle crossing task. This framework introduces a novel paradigm for quadruped robot control.

1. Introduction

Quadrupedal animals excel in fast, agile, and stable locomotion, with their diverse gaits enabling adaptation to various terrains. Whether sprinting up slopes or navigating dense forests, they move with remarkable flexibility. Inspired by these capabilities, researchers have developed quadruped robots with similar flexibility and adaptability. Pioneering studies include Spot by Boston Dynamics [1], ANYmal by ETH Zurich [2] and Cheetah by MIT [3]. Researchers use traditional control methods, including physics modeling and control theory [4,5], which been applied in various fields such as industrial inspection, disaster response, and exploration of unknown environments [6–8]. Although these traditional control methods have advantages in real-time performance and stability [9,10], it is still difficult to achieve stable locomotion control and precise locomotion planning in physics modeling with high-degree-of-freedom, and manual parameter adjustment by professionals is also required [11,12]. Consequently, further improving the stability and adaptability of quadrupedal robot locomotion and realizing automatic parameter adjustment have become the focus of current research.

In recent years, the development of embodied intelligence and physical intelligence has demonstrated that the behavior of a robot is not only affected by the control algorithm, but also depends on its own interaction with the environment [13–16]. This interactive learning mode increases the complexity of robot control, driving researchers to develop more efficient control algorithms. Specifically, deep reinforcement learning and imitation learning have been proven to be effective solution for addressing robot interaction with the environment, and these techniques have been widely applied in robot control [17–22]. Through deep reinforcement learning algorithms, robots interact with the environment, allowing them to autonomously learn optimal locomotion strategies without relying on precise physics modeling [23,24]. For example, Nicolas Heess *et al* [25] used a reward mechanism to obtain a forward locomotion strategy for a legged robot in the simulation environment, but the strategy failed to transfer from simulation to the real world. Building on this, Jie Tan *et al* [17] bridged the gap between simulation and reality by using a complex reward mechanism to train the controller and successfully transferred it to the real world in order to realize the agile movement of quadrupeds, however, this method performed poorly in reality. Joonho Lee *et al* [18] enabled quadrupedal

* Corresponding author.
E-mail address: song_jnu@jiangnan.edu.cn (Z. Song).

https://doi.org/10.1016/j.rineng.2025.104661
Received 11 December 2024; Received in revised form 6 March 2025; Accepted 17 March 2025
Available online 18 March 2025
2590-1230/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

robots to walk robustly in complex environment, but the adaptability is still insufficient in unknown terrains. Although deep reinforcement learning methods have made progress in the control of quadrupedal robots, they still face significant challenges in designing complex reward mechanisms, handling large parameter spaces, and transferring strategies from simulation to the real world [17,26,27].

To address the aforementioned challenges, imitation learning is applied as an alternative method for quadruped robot control. By mimicking the movement trajectory of an animal, robots can efficiently acquire motor skills, simplify the design of complex reward mechanisms, and facilitate the transfer of strategies from simulation to the real world [19,28–31]. For example, Peng X B *et al* [19] proposed a method to let a quadruped robot learn the walking ability by imitating the movement of animals. However, due to the single experimental scenario, it is not possible to obtain a large amount of data, which in turn limits the robot's ability to move on unknown terrain. In order to solve this problem, Yao Q *et al* [29] proposed a video-based imitation learning method to alleviate the problem of underfitting in the training process caused by insufficient data, but it still faces the challenges of complex data processing, limited imitation effect and poor environmental adaptability. Similarly, Li T *et al* 30] captured animal movement trajectory across various terrains, including flat ground, steep slopes, and stairs, for imitation learning, enhancing the robot's adaptability to diverse terrains. However, this method was limited by the data from specific terrains and lacked generalization capability on unknown terrains. Based on this, researchers combined imitation learning with reinforcement learning [32–34] to generate an initial policy by imitating expert dataset, and then use reinforcement learning to further optimize the policy, endowing the robot the ability to adapt to the new environment. Nonetheless, this approach has a strong dependence on the expert data set, reducing the flexibility of the control strategy. Relatively, Generative Adversarial Imitation Learning [35] trains an adversarial discriminator to distinguish between behaviors generated by the policy and those in the demonstration data. Through this adversarial training method, more realistic actions are generated. The disadvantage is that the generative algorithm is usually unstable and difficult to converge during the training process [36]. Recently, researchers have applied Adversarial Motion Prior (AMP) [37] to robot control, and improved the shortcomings of generative algorithms by using gradient penalty to reduce the oscillation during training. Besides, this method combines adversarial imitation learning with task reward, enabling the policy to mimic the behavior from the dataset while performing manually set tasks. The data-drive technique of adversarial motion prior has been applied to quadruped robots' motor skills [38], showing that the learned motion priors can encourage more natural and physically feasible behaviors, thus simplifying the migration of strategies from simulation to the real world. However, the locomotion of quadrupedal animals in nature follows the laws of physics, and over millions of years of evolution, their gaits and locomotor patterns have been highly optimized and adapted to variety of complex terrains [39,40]. By contrast, controller strategies obtained by learning-based methods may not accurately follow these physical laws, resulting in poor robot motion performance in the real world.

In this paper, we propose an innovative energy-efficient obstacle-crossing control framework for quadruped robots. By acquiring the obstacle characteristics and the robot's state, the framework can predict multiple feasible and lowest energy consumption obstacle-crossing trajectories, and control the robot to complete the obstacle crossing task. Based on this framework, we combine the data-driven technique of adversarial motion prior and the physics principle of energy consumption to design a jump controller and the obstacle-crossing trajectory prediction model, which not only effectively control the robot to cross obstacles, but also improve the efficiency of energy utilization. The main contributions are as follows:

1. In the proposed framework, we train a data-drive control strategy based on an adversarial motion prior that can control a quadruped robot to perform jumping maneuvers.
2. Based on physics, a trajectory prediction model is trained in the framework, which predicts multiple feasible and least energy-consuming obstacle-crossing trajectories by acquiring obstacle information and robot's current state.
3. The framework was successfully deployed on a real robot and completed the tasks of obstacle crossing and continuous jumping in the real world.

2. Methodology

2.1. Overview

In this chapter, we introduce the energy-efficient obstacle-crossing framework, as shown in Fig. 1. This framework consists of a jump controller and a trajectory predictor. The motion retargeting technology is used to process motion data, followed by setting the reward function and training the jump controller using the AMP algorithm in a simulation environment. Subsequently, obstacles of varying heights are set up in the simulation, and the quadrupedal robot is controlled to cross these obstacles with different motion parameters, during which motion data is collected to train the trajectory predictor through a neural network. Finally, the trajectory predictor generates multiple feasible and energy-efficient obstacle-crossing trajectories based on obstacle information and the robot's current state, and the jump controller controls the robot to complete the task. Detailed explanations of each part are provided below.

2.2. Jump controller module

2.2.1. Motion retargeting

As shown in Fig. 1 (b), the original motion capture dataset used is the German Shepherd motion capture data provided by Zhang H. *et al* 41], which includes quadruped walking, running, and jumping. To account for the morphological differences between animals and quadrupedal robots when using animal motion data, Peng X. B. *et al* 32] applied inverse kinematics to obtain joint angles, followed by forward kinematics to calculate the foot-end positions of the quadrupedal robot. Subsequently, the finite difference method was used to calculate joint velocities, base linear velocities, and angular velocities. These data were adapted to the A1 robot and used as samples for simulation training.

2.2.2. Reinforcement learning (RL)

The problem of legged locomotion in quadrupedal robots is modeled as a Markov Decision Process (MDP). MDP is a mathematical framework used to formally describe a series of decision-making problems in uncertain environments. In this study, MDP is defined as a 6-tuple $M = (S, A, f, r_t, p_0, \gamma)$, where S is the state space, representing the set of all possible states of the robot; A is the action space, representing all possible actions the robot can take; $f(s, a)$ is the system dynamics; $r_t(s, a, s')$ is the reward function. p_0 is the initial state distribution; and γ is the discount factor; The objective of RL is to find the optimal parameters θ for policy π_θ to maximize the expected discounted return, as mathematically expressed in Eq. (1).

$$J(\theta) = E_{\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t r_t\right] \tag{1}$$

2.2.3. Observation space

In the simulation experiments, the state space describes the state changes of the quadrupedal robot. In this paper, the state vector of the observation space is composed of $S_t = \{Sbase\ t, Sleg\ t, Scmd\ t, aleg\ t\text{-}1\} \in \mathbb{R}^{51}$, where the information of the body base is defined as $Sbase\ t = \{h, v,$

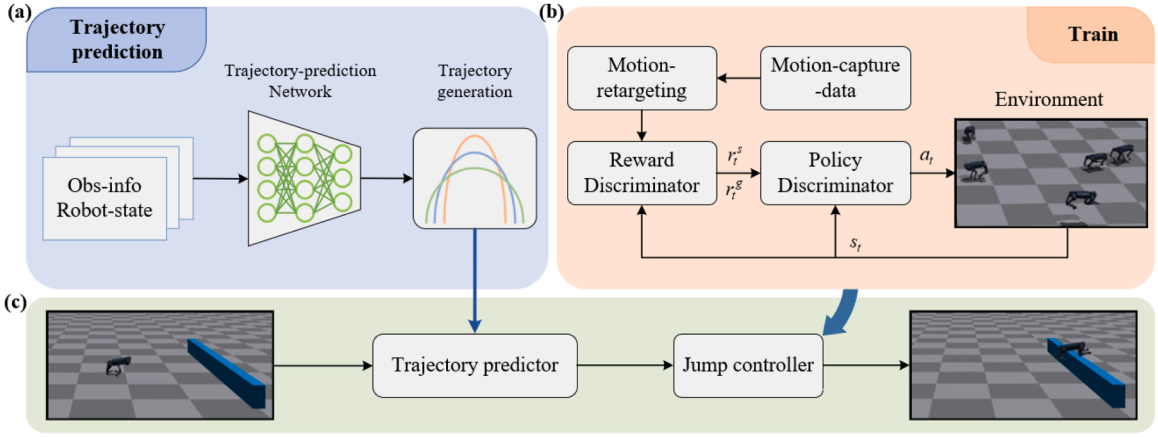


Fig. 1. Energy-saving obstacle-crossing control framework for quadrupedal robots. **(a)** Trajectory Predictor: Predicts obstacle-crossing trajectory based on obstacle information and robot state. **(b)** Jump Controller: Controls the quadrupedal robot to perform jumping actions. **(c)** Obstacle-Crossing Process: The trajectory predictor estimates the optimal obstacle-crossing trajectory based on obstacle information and the robot's current state, and passes the trajectory parameters to the jump controller, which controls the quadrupedal robot to complete the jumping action.

ω, $R\}\in\mathbb{R}^{10}$, the state of the leg is defined as *Sleg t* $=\{q_{leg}, \dot{q}_{leg}\}\in\mathbb{R}^{24}$, the control command is defined as *Scmd t* $=\{$*vcmd x, vcmd y, vcmd z,* $jump^{sig}$, $jump^{h}\}\in\mathbb{R}^{5}$, and leg action is defined as *aleg t-1*$\in\mathbb{R}^{12}$. Where $h\in\mathbb{R}^{1}$ is the height of the base, $v\in\mathbb{R}^{3}$ is the linear velocity of the base, $\omega\in\mathbb{R}^{3}$ is the angular velocity of the base, $R\in\mathbb{R}^{3}$ is the orientation of base, $q_{leg}\in\mathbb{R}^{12}$ is the position of the leg joints, $\dot{q}_{leg}\in\mathbb{R}^{12}$ is the velocity of the leg joints, *vcmd* $x\in\mathbb{R}^{1}$ is the x-axis linear velocity, *vcmd* $y\in\mathbb{R}^{1}$ is the y-axis linear velocity, *vcmd* $z\in\mathbb{R}^{1}$ is the z-axis linear velocity, $jump^{sig}\in\mathbb{R}^{1}$ is the jump signal, and $jump^{sig}\in\mathbb{R}^{1}$ is the jump height.

2.2.4. Action space

The action space, as the output of the neural network, directly controls the locomotion changes of the quadrupedal robot. In this paper, the action space $A\in\mathbb{R}^{12}$ represents the motor torques at the robot's joints. These torques are derived from the target joint angles through a proportional–derivative (PD) controller, with the detailed expression shown in Eq. (2):

$$\tau = K_p \times \left(q_t^* - q_t\right) + K_d \times \left(-\dot{q}_t\right) \tag{2}$$

where τ represents the torque, K_p is the proportional gain, K_d is the derivative gain, $q_t^*$is the desired joint position, $q_t$is the current joint position, and $\dot{q}_t$is the joint velocity.

2.2.5. Reward function

To ensure the quadrupedal robot to better learn the walking, running and jumping motion of quadruped in the simulation environment, the reward function consists of an adversarial motion prior style reward *rs t* and a task reward *rg t*. The style reward encourages the robot to mimic behaviors from the expert dataset, while the task reward motivates the robot to track the jumping height, angular velocity, and linear velocity at time step *t*. The task reward function is given in Eqs (3) to (6):

$$r_t^{g} = w^{v} r_t^{lin_tra} + w^{\omega} r_t^{ang_tra} + w^{h} r_t^{h} \tag{3}$$

$$r_t^{lin_tra} = \exp\left(-5\left\|\widehat{v}_t^{xy} - {v_t}^{xy}\right\|^2\right) \tag{4}$$

$$r_t^{ang_tra} = \exp\left(-3.3\left\|\widehat{\omega}_t^{z} - {\omega_t}^{z}\right\|^2\right) \tag{5}$$

$$r_t^{h_tra} = \exp\left(-5\left\|\widehat{h}_t^{z} - {h_t}^{z}\right\|^2\right) \tag{6}$$

where w^{v}=0.5, w^{ω}=0.5 and w^{h}=0.8 are weighting coefficients that control the influence of each term in the reward function. To maximize the robot's jumping height, w^{h} is assigned the highest relative value.$\widehat{v}_t^{xy}$ represents the desired linear velocity in the *xy*-plane, $\widehat{\omega}_t^{z}$represents the desired angular velocity along the *z*-axis, and $\widehat{h}_t^{z}$ represents the desired height of the center of mass along the *x*-axis.

2.2.6. Network structure

In this study, an Actor network is implemented using a Multilayer Perceptron (MLP) with a structure of [512, 256, 128, 12], and the Exponential Linear Unit (ELU) is employed as the activation function. The input to the network is the dimension of the quadrupedal robot's observation state, and the output is the target joint angles. Additionally, a Critic network is designed with a structure of [512, 256, 128, 1] to enhance the learning process. A discriminator D_ϕ is defined to distinguish whether the state transition (*s, s'*) is a real sample from the dataset or a fake sample generated by the policy. The discriminator and style reward functions proposed by Peng X. B. *et al* 37] are adopted, with the corresponding formulas presented in Eqs. (7) and (8):

$$\begin{aligned}&\arg_{\phi}\min E_{(s,s')\sim D}\left[\left(D_\phi(s,s')-1\right)^2\right]\\&+E_{(s,s')\sim\pi_\theta(s,a)}\left[\left(D_\phi(s,s')+1\right)^2\right]\\&+\frac{w^{gp}}{2}E_{(s,s')\sim D}\left[\left\|\nabla_\phi D_\phi(s,s')\right\|^2\right]\end{aligned} \tag{7}$$

where the first two terms are designed to encourage the discriminator to distinguish whether the input state transition (*s, s'*) is from the reference dataset or generated by the policy. The last term is a gradient penalty, which penalizes samples with non-zero gradients in the dataset, with a coefficient w^{gp} of 10.

$$r_t^{s}(s_t, s_{t+1}) = \max\left[0, 1 - 0.25\left(D(s,s')-1\right)^2\right] \tag{8}$$

Where additional offsets, scaling and cropping are applied to limit the reward to between [0, 1], in order to maintain algorithmic stability and ensure a uniform range of reward values.

2.2.7. Implementation of the adversarial motion prior

As shown in Fig. 1 (b), this study constructed a jump controller model based on the adversarial motor prior. First, the data from motion capture was converted into the data format required for reinforcement learning training using motion retargeting technique. In order to enable the quadrupedal robot to learn the walking, running and jumping behaviors of the quadrupeds in the dataset more efficiently, the task reward function *rg t* and the style reward function *rs t* are designed in this paper. During the training process, the robot performs the action a_t in

the environment and generates the state s_t. This state is used in two computational processes: on one hand, the input discriminator is used to compute the style reward *rs t*, and on the other hand, it is used to compute the task reward *rg t*. Subsequently, *rs t, rg t* and s_t work together to optimize the discriminator and the policy. After the policy is updated, the robot performs the next action a_{t+1}, which generates a new state s_{t+1}, and the process continues until the combined reward values of *rs t* and *rg t* converge to the optimal and stabilize. Eventually, the trained policy not only integrates the walking, running and jumping behaviors in the dataset, but also achieves a smooth transition between different gaits, thus constructing a stable and efficient jumping controller.

2.3. Trajectory predictor module

In the simulation environment, although the quadrupedal robot can be manually guided to perform obstacle-crossing tasks through the jump controller, it lacks the ability to autonomously complete these tasks. To eliminate the influence of manual control on task execution and enhance the quadrupedal robot's obstacle-crossing decision-making ability during running, a trajectory predictor is introduced as a preliminary stage of the jump controller, as shown in Fig. 1 (c). The trajectory predictor can predict multiple feasible and energy-efficient obstacle-crossing trajectories based on obstacle information and the robot's current state, as shown in Fig. 1 (a), and then the jump controller controls the quadruped robot to complete the jumping action. This trajectory predictor consists of the Jump Trajectory Prediction Network (Jtp Net) and the Minimum Energy Consumption Prediction Network (Mecp Net).

2.3.1. Jtp Net

The network processes a four-dimensional input vector, which includes the takeoff velocity v, takeoff distance l, maximum center-of-mass height h during obstacle crossing, and takeoff angle θ. The output corresponds to the center-of-mass coordinates for the takeoff, flight, and landing phases of the quadrupedal robot's jump. The network structure is detailed in Table 1. The loss function is the Mean Squared Error (MSE), with its specific definition provided in Eq. (9):

$$\mathrm{L}(u,t)=\frac{1}{n}\sum_{i=1}^{n}(u_i-t_i)^2 \tag{9}$$

Table 1
Structure of JTP Net and Mecp Net.

Layer	Structure	Jtp Net Parameters	Mecp Net Parameters
Layer1	Batch Normalization LeakyReLU	Input, output = (4,128) Dropout=0.5 Negative slope=0.01	Input, output = (6,128) Dropout=0.5 Negative slope=0.01
Layer2	Batch Normalization LeakyReLU	Input, output = (128,256) Dropout=0.5 Negative slope=0.01	Input, output = (128,256) Dropout=0.5 Negative slope=0.01
Layer3	Batch Normalization LeakyReLU	Input, output = (256,128) Dropout=0.5 Negative slope=0.01	Input, output = (256,128) Dropout=0.5 Negative slope=0.01
Layer4	Batch Normalization LeakyReLU	Input, output = (128,64) Dropout=0.5 Negative slope=0.01	Input, output = (128,64) Dropout=0.5 Negative slope=0.01
Layer5	Batch Normalization LeakyReLU	Input, output = (64,3) Dropout=0.5 Negative slope=0.01	Input, output = (64,3) Dropout=0.5 Negative slope=0.01

where u represents the input, including v, l, h and θ, n is the number of samples, u_i is the predicted value, and t_i is the target value.

2.3.2. Mecp Net

The network processes a six-dimensional input vector consisting of obstacle height H and width W, takeoff velocity v, takeoff distance l, maximum center-of-mass height h during obstacle crossing, and takeoff angle θ. The output represents the center-of-mass coordinates for the takeoff, flight, and landing phases of the quadrupedal robot's jump with minimal energy consumption. The network structure is detailed in Table 1. To comprehensively evaluate prediction accuracy and energy efficiency, a composite loss function is defined, comprising the Mean Squared error ($\mathrm{MSE_{dim}}$) loss and the energy loss ($\mathrm{E_{dim}}$), both of which are dimensionally normalized to ensure consistency in magnitude. The relevant mathematical formulas are provided in Eqs (10) to (14):

$$\mathrm{L}(y,t)=\alpha\cdot\mathrm{MSE_{dim}}+\beta\cdot\mathrm{E_{dim}} \tag{10}$$

$$\mathrm{MSE}(y,t)=\frac{1}{n}\sum_{i=1}^{n}(y_i-t_i)^2 \tag{11}$$

$$\mathrm{E}_{mech}(y)=\frac{1}{n}\sum_{i=1}^{n}\left[g\cdot(h_i-\overline{h})+0.25\cdot\left(\frac{g\cdot l_i^2}{h_i-\overline{h}}-v_i^2\tan^2\theta\right)\right] \tag{12}$$

$$\mathrm{MSE_{dim}}=\frac{\mathrm{MSE}(y,t)}{\frac{1}{n}\sum_{i=1}^{n}t_i^2}=\frac{\frac{1}{n}\sum_{i=1}^{n}(y_i-t_i)^2}{\frac{1}{n}\sum_{i=1}^{n}t_i^2} \tag{13}$$

$$\mathrm{E_{dim}}=\frac{\mathrm{E_{mech}}(y)}{\overline{\mathrm{E}}_{\mathrm{mech}}(y)} \tag{14}$$

where $\mathrm{E_{mech}}$ [42] reflects the potential energy change of the center of mass of the robot, as well as the kinetic energy change in the process of takeoff and obstacle crossing. A smaller $\mathrm{E_{mech}}$ value indicates lower energy loss. The input y includes parameters H, W, v, l, h and θ, where $\overline{h}$=0.35 represents the average center-of-mass height during the robot's locomotion. Through the experimental test, when α=0.8 and β=0.2 are the equilibrium points that meet the two loss terms of simultaneous optimization. g is the gravitational acceleration, n is the number of samples, y_i is the predicted value, and t_i is the target value.

2.4. Sim-to-real transfer

To narrow the gap between simulation and the real world, the domain randomization method [43] was used to transfer the learned control strategy from simulation to reality. In the simulation environment, ground friction coefficients, robot base mass, and PD control gains were randomized. During training, random velocity perturbations were applied to the robot in different directions at fixed time intervals. The randomized variables and the uniform distribution ranges for sampling are shown in Table 2.

2.5. Implementation of control framework

Through the design presented in Sections 2.2 and 2.3 of this chapter,

Table 2
Randomized simulation parameters.

Parameters	Randomization Range
Friction	[0.25,1.75]
Added Base Mass	[-1.0,1.0] kg
Push Interval	15 s
X-axis Velocity Push	[-1.0,1.0] m/s
Y-axis Velocity Push	[-1.0,1.0] m/s
Motor Stiffness Gain Multiplier	[0.9,1.1]
Motor Damping Gain Multiplier	[0.9,1.1]

this study has completed the jump controller model based on adversarial motion prior and the trajectory predictor model based on data fitting. The following provides a detailed introduction to the integration of these two models.

As shown in Fig. 1 (a), the trajectory predictor obtains the height H and width W of the obstacle, as well as the distance l between the robot and the obstacle and the robot's velocity v, and predicts multiple feasible trajectories based on this information. Each trajectory contains information about the key positions of the robot during the jumping process, including the takeoff point, the highest point during the flight phase, and the landing point, and all the positions are referenced to the obstacle. For each trajectory, the trajectory predictor gives the corresponding jump angle θ and jump height h. As shown in Fig. 1 (c), the trajectory predictor takes the obtained θ and h as inputs and passes them to the jump controller to control the quadruped to complete the jumping action of the corresponding height in accordance with a certain angle. Ultimately, the control framework can select the lowest energy consumption or other safe and feasible jumping trajectories according to the actual situation to ensure the stability and adaptability of jumping.

3. Experimental results and discussion

3.1. Experimental conditions

3.1.1. Experimental setup

1)Robot model: In this work, Unitree A1 robot was used as an experimental model with robot related parameters as shown in Table 3. The training and simulation validation is done in the simulation environment Isaac-Gym using the AMP algorithm.

2)Nature of obstacles: The obstacles used in the simulation environment are rectangular bodies with collision properties, which have a length of 50cm, a width range of 18cm to 22cm, and a height range of 15cm to 55cm.

3.1.2. Energy evaluation metric

The Cost of Transport (COT) is used as the evaluation metric of energy consumption when quadruped robot cross obstacles. COT can compare energy consumption between robots and control strategies with different sizes, weights or dynamic characteristics, and is a key dimensionless metric to evaluate the locomotion efficiency of robots. In this study, COT is utilized to measure the energy consumption of a quadruped robot during obstacle crossing. COT is defined as the ratio of power to the product of weight and velocity, which is mathematically expressed as Eq. (15):

$$\mathrm{COT} = \frac{\mathrm{Power}}{\mathrm{Weight} \times \mathrm{Velocity}} = \frac{\sum_{actuators}[\tau \cdot \dot{\theta}]}{W \times \| v \|} \tag{15}$$

where τ represents the joint torque, $\dot{\theta}$ is the motor velocity, W is the robot weight, and $||v||$ is the robot velocity. The robot's energy efficiency during the obstacle-crossing process can be evaluated by quantifying the sum of the products of joint torque and motor velocity. The robot's weight and velocity provide standardization, allowing for efficiency comparisons under different control parameters.

Table 3
Robot parameters.

Parameters	Value
Max Joint Torque	33.5 Nm
Max Joint Speed	21 Rad/s
Whole Machine Weight	12 kg
Length Width Height	500m*300m*400m
Hip Joints Angle Range	[-46°,46°]
Thigh Joints Angle Range	[-60°,-240°]
Knee Joints Angle Range	[-154.5°,-52.5°]

2.1.3. Training setup

A total of 6,000 agents were trained in parallel across 12,000 episodes in the simulator. An episode would terminate and reset the robot if the robot's trunk collided with the ground or if it maintained a posture for an extended period of time. The entire training was conducted on a desktop computer equipped with an Intel(R) Xeon(R) Platinum 8175M CPU @ 2.50GHz processor and an NVIDIA GeForce RTX 3090 GPU with 24GB of memory. The algorithm's training time was approximately 8 hours.

3.1.4. Deployment

When deploying the control strategy from simulation to the real world, a laptop was used as the upper-level controller, communicating with the robot's local network via an Ethernet cable. The Unitree official SDK was utilized for data transmission, with a control frequency of 67Hz.

3.2. Trajectory generation summary

As shown in Fig. 2 (a), in the simulation environment, the quadrupedal robot crossed obstacles of varying heights at different velocities, illustrating the relationship between takeoff velocity and center-of-mass height. In order to just cross the obstacle, the velocity increases as the height of the obstacle increases. At the same time, the center of mass height also tends to increase positively with the increase in the height of the obstacle. Further observation shows that as the obstacle height increases during crossing, the vertical distance between the center of mass and the top of the obstacle gradually decreases. This phenomenon indicates that the designed jump controller has strong generalization capabilities, as the quadrupedal robot adopts different swing leg strategies when facing obstacles of varying heights, rather than using the same posture, which results in varying distances between the center of mass and the obstacle.

To further validate this phenomenon, the postures of the quadrupedal robot during takeoff, flight, and landing when crossing low and high obstacles were visualized. By comparing the posture diagrams in Fig. 2 (b) with **(e), (c)** with **(f)**, and **(d)** with **(g)**, we can further analyze the robot's locomotion strategies when dealing with obstacles of different heights. When facing lower obstacles, as shown in Fig. 2 (b), the angle between the hind legs' shank and thigh during the takeoff phase is relatively small, indicating that the robot does not need to release much energy to cross lower obstacles. In the flight phase in Fig. 2 (c), the swing amplitude of the hind legs is smaller, and the angle between the legs and the body is larger, which lowers the center of mass and helps maintain dynamic balance. In the landing phase in Fig. 2 (d), the robot adopts an energy-recovery posture to reduce impact and minimize energy loss upon landing. Conversely, when facing higher obstacles, as shown in Fig. 2 (e) during the takeoff phase, the angle of the hind legs significantly increases, indicating that more energy needs to be released to achieve a larger swing of the hind legs for takeoff. During the flight phase, as shown in Fig. 2 (f), the swing amplitude of the hind legs increases, and the legs tuck more closely to the body, facilitating vertical lift of the robot's center of mass to cross higher obstacles. In the landing phase, shown in Fig. 2 (g), the robot also adopts an energy-recovery posture to minimize impact and energy loss during landing.

Based on the simulation results and visual analysis, the quadrupedal robot adopted different locomotion strategies when facing obstacles of varying heights, demonstrating the adaptability of its control system. As the obstacle height increased, the robot adjusted its takeoff velocity and posture to optimize energy usage during the obstacle-crossing process. These diverse locomotion strategies not only improved the robot's obstacle-crossing ability but also demonstrated its precise control over energy management and dynamic balance in complex environments.

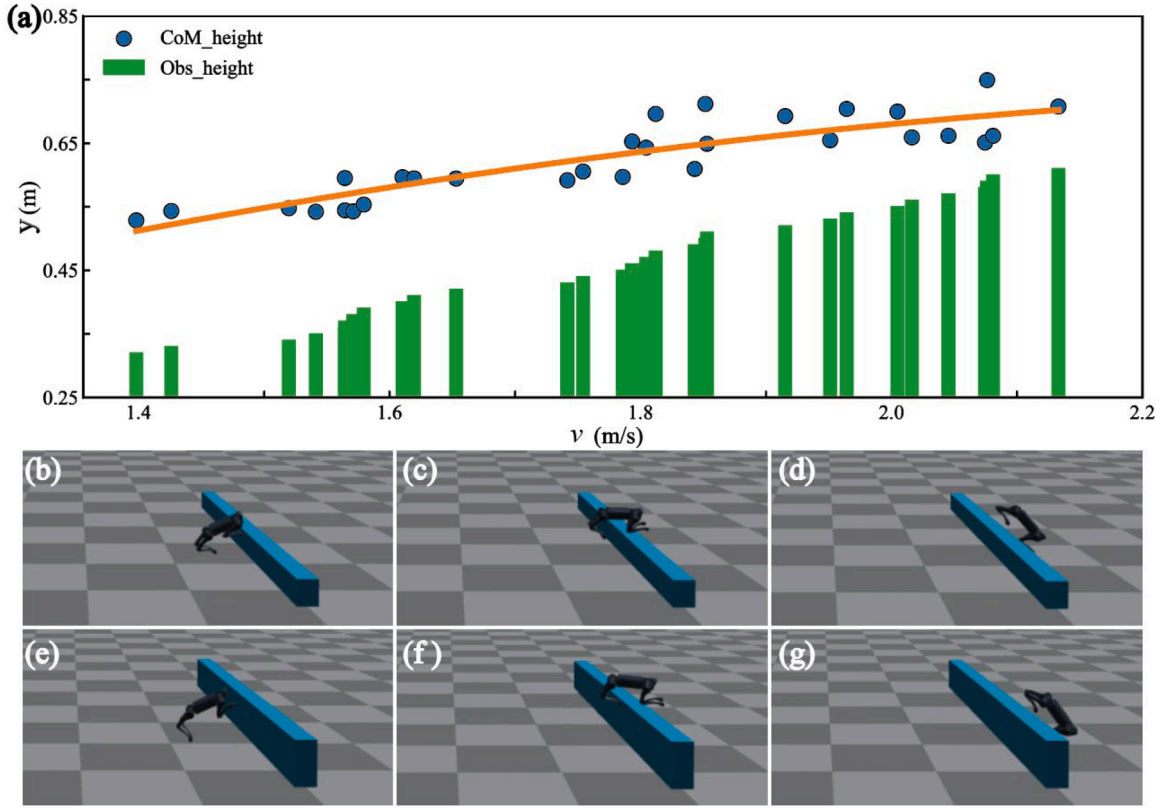


Fig. 2. Dynamic parameters and motion postures of the quadrupedal robot while crossing obstacles of different heights. **(a)** Velocity and center-of-mass height when just crossing obstacles of different heights **(b-d)** Takeoff, flight, and landing postures while crossing a 0.3m obstacle. **(e-g)** Takeoff, flight, and landing postures while crossing a 0.5m obstacle.

3.3. Energy consumption evaluation

The action sequence analysis in Fig. 3 (a) shows that the first frame represents the takeoff phase, during which the hind legs compress to store kinetic energy. The second and third frames represent the flight phase, where the robot's limbs extend to adjust its mid-air posture. The final frame illustrates the landing phase, where the robot's forelimbs contact the ground to dissipate the impact. The velocity tracking in Fig. 3 (b) clearly shows that velocity increases during the takeoff phase, indicating that the robot is storing kinetic energy to cross the obstacle. From takeoff to flight, the velocity remains relatively stable, demonstrating the control system's effectiveness in maintaining a consistent velocity. The energy consumption data in Fig. 3 (c) shows that the COT peaks during the jump phase and gradually decreases as the robot stores energy during the jump and releases it after takeoff. As shown in Table 4, the obstacle-crossing trajectory generated by the Minimum Energy Consumption Prediction Network shows the lowest COT peak during the takeoff and landing phases, indicating that the network successfully produced the most energy-efficient jumping strategy. Energy consumption rises again during the landing phase as energy is required to absorb the impact of landing.

As shown in Fig. 3 (d), the robot's real velocity maintains a small deviation from the commanded velocity throughout the motion, and the velocity profile is smooth without violent fluctuations, indicating that the control framework has good velocity tracking capability and stability. This is further verified in Fig. 3 (e), where the robot's center of mass trajectory remains smooth before and after crossing the obstacle without sudden changes, indicating that the robot attitude is stable. Meanwhile, the trajectories of the forefoot and rearfoot maintain a regular motion rhythm before and after crossing the obstacle, and there are no abnormal and drastic changes during the obstacle crossing process, reflecting the reasonableness of the gait planning and the stability of the support.

In this paper, the minimum energy consumption of a quadruped animal [42] when crossing an obstacle is used as a reference and compared with the minimum energy consumption of the quadruped robot proposed in this paper when crossing an obstacle. As shown in Table 5, the comparison results show that the energy consumption of the quadruped robot in this paper's method when crossing obstacles differs from that of the quadruped robot by about 5% ~ 8%, and the minimum difference is only 5.58%. This shows that the proposed method makes the quadruped robot very close to the optimized level of energy consumption achieved by quadrupeds after millennia of natural evolution in terms of energy-saving performance in obstacle crossing, thus further verifying the effectiveness of the proposed method.

By analyzing the energy consumption and motion strategy of a quadruped robot during obstacle crossing, this paper fully demonstrates the superiority of the proposed control framework. The framework can not only efficiently control the robot to cross obstacles, but also optimize the energy consumption to minimize the energy consumption during jumping and landing. Moreover, the proposed framework makes the energy consumption performance of the quadruped robot during obstacle crossing very close to that of a quadruped animal. In addition, the robot is able to maintain a stable velocity and body attitude during obstacle crossing, which improves the smoothness of the overall motion. However, as shown in Table 6, the shortcoming of this framework is that compared to the obstacle-crossing task at the same speed, the time to complete the task increases by 28.8% when using the energy-efficient obstacle-crossing framework, which is mainly limited by the computational overhead of the energy model solution. This is where future work needs to be improved to increase computational efficiency and reduce the time required for obstacle crossing.

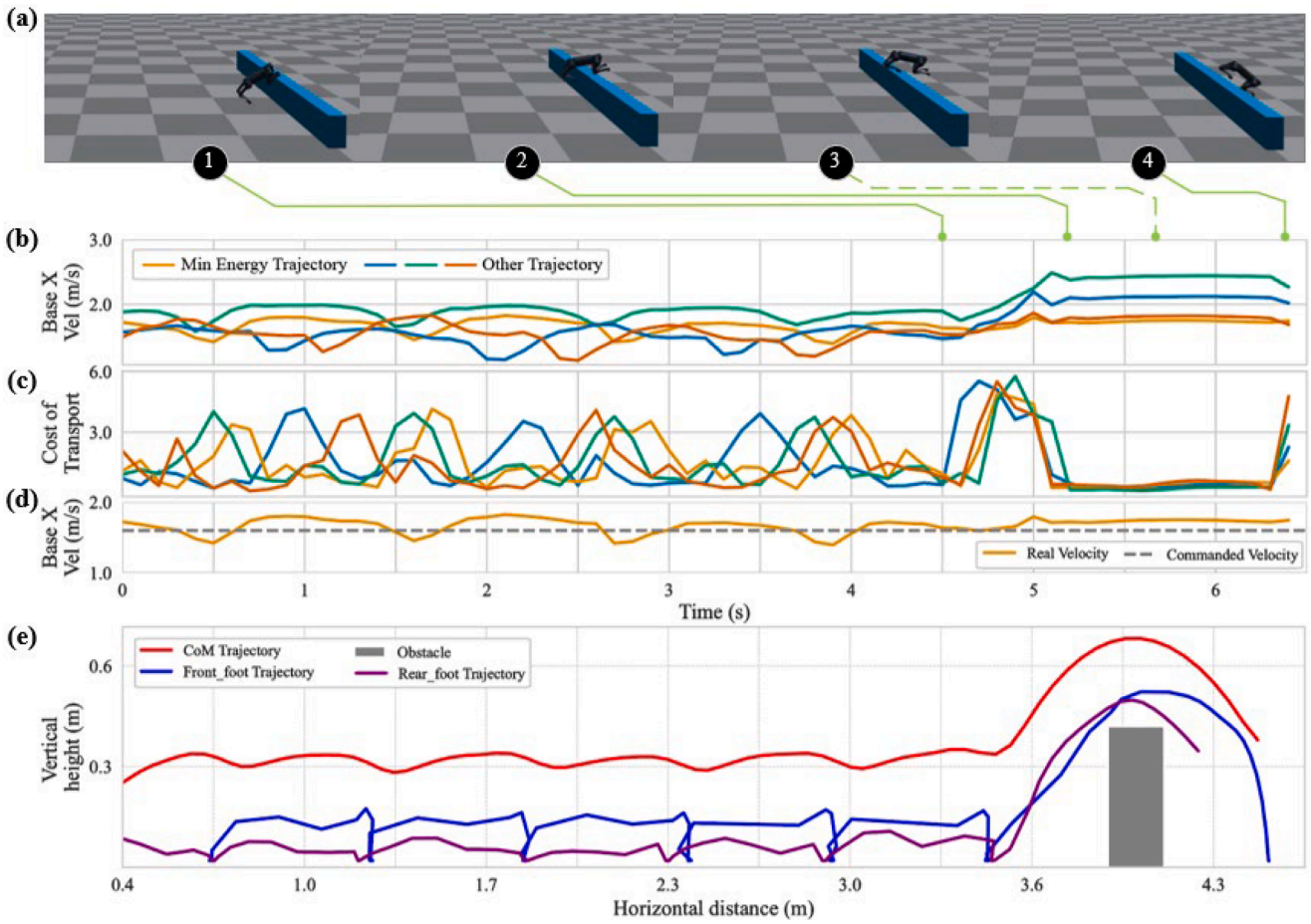


Fig. 3. Velocity tracking and energy consumption of a quadruped robot during crossing a 0.4m obstacle with different control parameters. **(a)** Continuous action sequence during takeoff, flight, and landing phases. **(b)** Velocity changes in the x-axis direction during the approach and obstacle-crossing phases. **(c)** Estimated COT during the approach and obstacle-crossing phases. **(d)** Performance of velocity tracking in the X-axis direction. **(e)** The trajectory of the foot end and the centroid of the robot during obstacle crossing.

Table 4
Comparison of COT peaks for different trajectories when crossing a 0.4m obstacle.

	Take-off COT Peak	Landing COT Peak
Trajectory 1	5.77	3.34
Trajectory 2	5.58	2.28
Trajectory 3	5.47	4.76
OUR	**4.90**	**1.60**

Table 5
Comparison of Energy Consumption for Robot and Animal Obstacle Crossing.

Height of obstacles (m)	Quadruped robot energy consumption (J/KG)	Quadruped animal energy consumption (J/KG)	Difference in energy consumption
0.35	3.36	3.12	**7.69%**
0.4	4.73	4.48	**5.58%**
0.48	5.03	4.76	**5.67%**
0.55	6.15	5.78	**6.40%**

Table 6
Computational efficiency.

Method	Obstacle crossing time (s)	Extra time
AMP	5.2	
OUR	**6.7**	**+28.8%**

3.4. Motion evaluation and strategy selection

As shown in Fig. 4 (a), the simulation data comes from experiments where the quadrupedal robot crossed obstacles of different heights at varying velocities in a simulated environment. By inputting four parameters into the Jtp Net: takeoff velocity v, distance from the obstacle l, maximum center-of-mass height h, and takeoff angle θ, the network can predict the center-of-mass coordinates at each stage of the obstacle-crossing process, including coordinates A during the takeoff phase, B during the flight phase, and C during the landing phase. These coordinates are used to determine the center-of-mass trajectory of quadrupedal robot during obstacle crossing. In the Mecp Net, by inputting six parameters: obstacle height H, obstacle width W, takeoff velocity v, distance from the obstacle l, maximum center-of-mass height h, and takeoff angle θ, the network predicts the center-of-mass trajectory that minimizes energy consumption during the obstacle-crossing process. As shown in Fig. 4 (a), eight sets of random parameters are input to the Jtp Net, and the parameters range within the control parameters of the quadruped robot crossing the obstacles 0.45 meters high and 0.2 meters wide. One set of parameters represents the obstacle crossing trajectory with the lowest center of mass, which will be used to determine the obstacle information. The dashed lines in Fig. 4 (b) represent multiple feasible center-of-mass obstacle-crossing trajectories generated by the Jtp Net based on different parameter combinations. Subsequently, by inputting the obstacle information and robot state into the Mecp Net, the center-of-mass trajectory with the lowest energy consumption can be predicted. In Fig. 4 (c), the front and hind foot

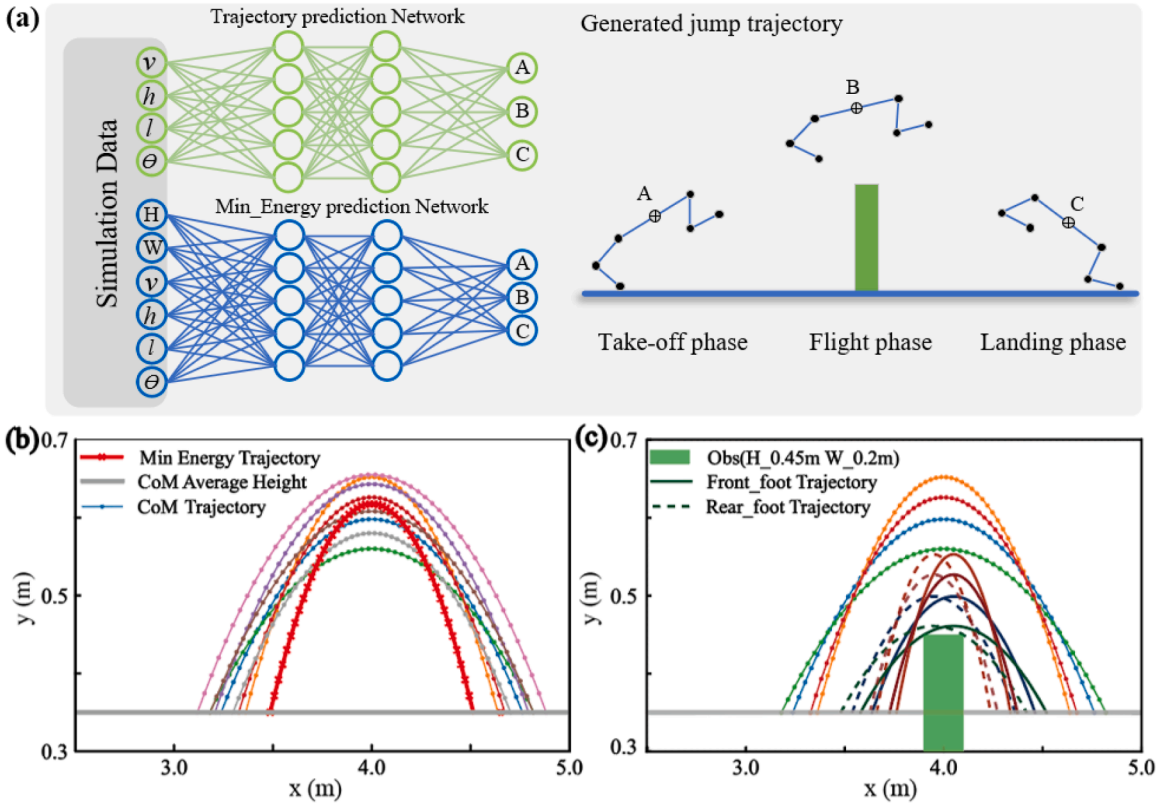


Fig. 4. Quadrupedal robot jumping trajectory prediction. **(a)** Composition of the jump trajectory predictor: Jump Trajectory Prediction Network and Minimum Energy Consumption Prediction Network. **(b)** The predicted center-of-mass trajectories based on the prediction network. The dashed lines represent multiple center-of-mass trajectories that can cross the same obstacle, with the bold red line indicating the trajectory with the least energy consumption required for crossing the obstacle. The gray line represents the average center-of-mass height during the robot's approach. **(c)** Multiple predicted center-of-mass trajectories are used to derive the front and hind foot-end trajectories. The obstacle information is determined based on these foot-end trajectories, with the solid line representing the front foot trajectory, the dashed line representing the hind foot trajectory, and the green rectangle representing the obstacle.

trajectories are derived from the predicted center-of-mass trajectory. The dimensions of the obstacle are determined based on the relationship where the front foot trajectory is tangent to the left side of the obstacle's highest point, and the hind foot trajectory is tangent to the right side of the obstacle's highest point as the quadrupedal robot just crosses the obstacle. As shown in Fig. 4 (c), the obstacle height is determined to be 0.45 meters and the width to be 0.2 meters, verifying the accuracy of the prediction model.

The results demonstrate that by acquiring obstacle information and robot state, Mecp Net can generate multiple feasible trajectories and corresponding jump parameters, which are used as inputs to the jump controller, enabling the quadrupedal robot to successfully cross obstacle. When the quadrupedal robot is running in the field and an obstacle suddenly appears in the blind area of the field of vision, the control strategy will select the appropriate takeoff angle and height according to the robot's current velocity and the distance to the obstacle to ensure successful crossing the obstacle. If the obstacle is still some distance away from the robot, the control strategy can predict the obstacle-crossing trajectory with the lowest energy consumption. This enhances the robot's adaptability in the field while also improving energy efficiency.

3.5. Performance of the strategy in the real world

In addition to the evaluation of the control strategy in the simulation environment, we also deployed it on a real quadruped robot. As shown in Fig. 5 (a-c), the robot successfully crossed an obstacle up to 16 cm high under the control of the strategy, demonstrating its obstacle-crossing capability. Fig. 5 (d-i) show that the robot can perform consecutive jumps of up to 18 cm under the guidance of the strategy. After the first jump, the robot quickly adjusts its posture and stably performs the second jump. Fig. 5 (j-l) demonstrate that the robot can perform vertical jumps of up to 27 cm in place.

By deploying the strategy on a real quadrupedal robot, we verified its effectiveness in real world. The test results showed that, under the guidance of the strategy, the robot was able to effectively cross obstacles and successfully complete consecutive jumping tasks. This demonstrates that the strategy not only performed well in the simulation environment but also exhibited good adaptability and stability in real-world applications, providing support for the further development and application of quadrupedal robots.

4. Conclusion and future work

Our work demonstrates that the energy-efficient obstacle-crossing control framework can improve the obstacle-crossing capability and energy utilization of quadrupedal robots in complex terrains. Specifically, by combining the data-driven method of adversarial motion prior and the physical law of animal motion, we designed a unified control framework including a jump trajectory predictor and a jump controller. With this framework, the robot only requires obstacle information and its current state to predict multiple feasible and energy-efficient obstacle-crossing trajectories, enabling it to accurately control the robot to successfully cross obstacles.

This study proposes a method that effectively enhances the adaptability and energy optimization of quadruped robots in outdoor environments. However, the current environmental perception mechanism still has certain limitations: the robot mainly relies on proprioception for

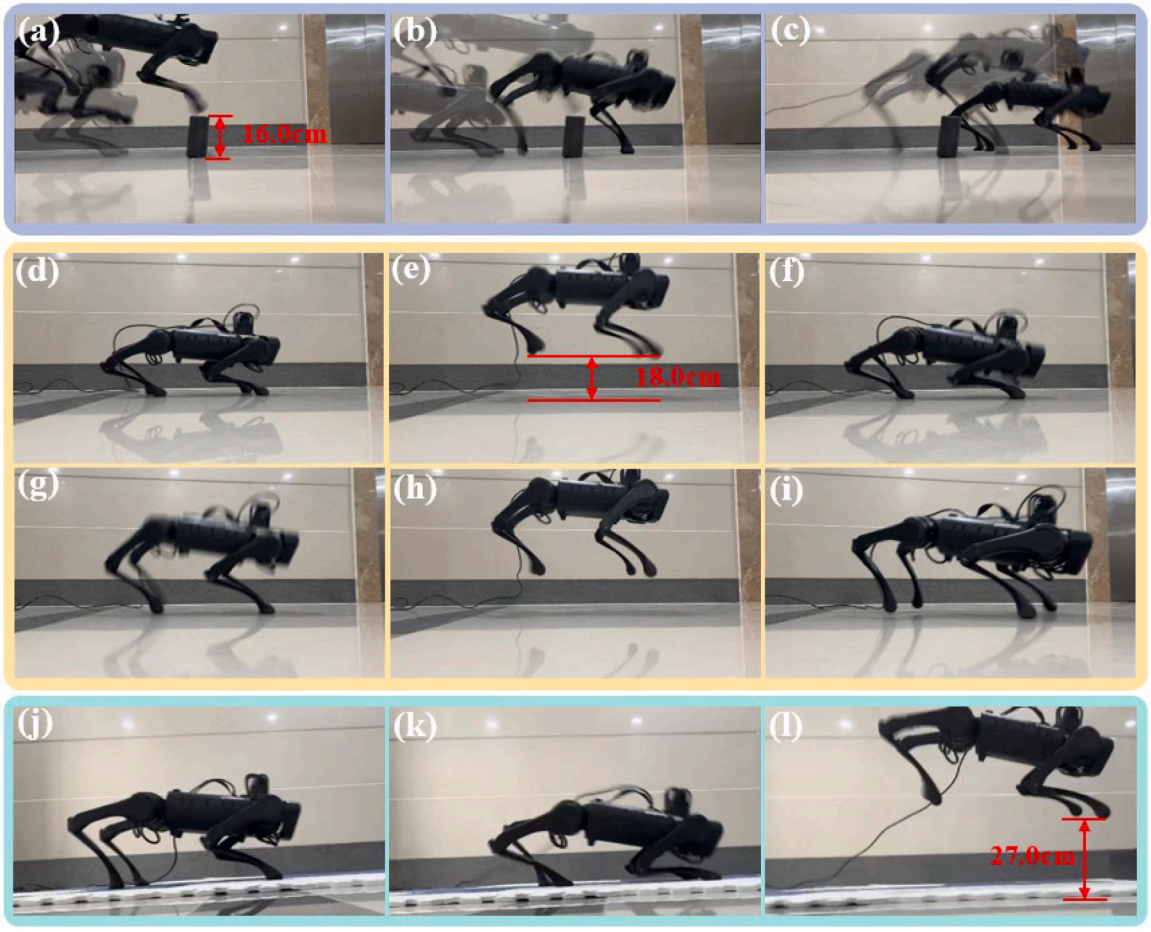


Fig. 5. Performance of the control strategy deployed on the real robot. **(a-c)** The robot successfully crosses an obstacle with a height of 16 cm. **(d-i)** The robot can complete a continuous jump with a height of about 18cm. **(j-l)** The robot achieves a jump height of up to 27 cm in place.

environmental adaptation, and obstacle information needs to be manually pre-programmed. This dependence on prior information limits its adaptability in diverse terrains or in highly unpredictable environments. To overcome this limitation, future research could explore integrating drone-assisted environmental perception to further enhance the robot's adaptability across diverse terrains in outdoor environments. In addition, in terms of obstacle-crossing strategies, this paper focuses on jumping gait, while crossing is more energy efficient for lower obstacles. Future research can combine multi-sensor information, such as LIDAR and stereo camera, to increase the robot's environment sensing ability and explore the optimal obstacle-crossing strategy under different terrain and obstacle conditions, so that the robot can autonomously choose a more efficient gait. This will further enhance the robot's adaptability in complex environments and strike a better balance between energy saving and motion efficiency. Finally, in order to enhance the practical application value of this method, future usability tests are needed to evaluate its performance in real complex environments and to explore its potential applications in collaborative robotics (Cobot) scenarios. For example, in search and rescue, environmental monitoring, and disaster response tasks, quadruped robots not only need to cross obstacles autonomously, but also may need to work together with humans or other robots. Therefore, future research could further optimize the human-robot interaction interface and enhance the interpretability and reliability of the control framework to ensure the robot's efficient collaborative capability in variable environments.

CRediT authorship contribution statement

Jiang Han: Writing – review & editing, Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Baishu Wan:** Writing – review & editing, Software, Data curation. **Yilin Zheng:** Writing – review & editing, Software, Data curation. **Zhigong Song:** Writing – review & editing, Supervision, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 12202159) (Grant No. 12472216), the Postgraduate Research & Practice Innovation Program of Jiangsu Province (Grant No. SJCX24_1350) and the Wuxi Taihu Lake Talent Plan. We extend our gratitude to the School of Mechanical Engineering at Jiangnan University for providing computational resources.

Data availability

Data will be made available on request. The data and code supporting the findings of this study are available from the authors upon

reasonable request.

References

[1] S. Zimmermann, R. Poranne, S. Coros, Go fetch!-dynamic grasps using boston dynamics spot with external robotic arm, in: 2021 IEEE International Conference on Robotics and Automation (ICRA), , IEEE, 2021, pp. 4488–4494, https://doi.org/10.1109/icra48506.2021.9561835.

[2] M. Hutter, C. Gehring, D. Jud, A. Lauber, C D Bellicoso, V. Tsounis, J. Hwangbo, K. Bodie, P. Fankhauser, M. Bloesch, Anymal-a highly mobile and dynamic quadrupedal robot, in: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2016, pp. 38–44, https://doi.org/10.1109/IROS.2016.7758092.

[3] S. Seok, A. Wang, M Y Chuah, D. Otten, J. Lang, S. Kim, Design principles for highly efficient quadrupeds and implementation on the MIT Cheetah robot, in: 2013 IEEE International Conference on Robotics and Automation, IEEE, 2013, pp. 3307–3312, https://doi.org/10.1109/ICRA.2013.6631038.

[4] H. Miura, I. Shimoyama, Dynamic walk of a biped, Int. J. Rob. Res. 3 (2) (1984) 60–74, https://doi.org/10.1177/027836498400300206.

[5] U. Saranli, M. Buehler, D.E. Koditschek, RHex: a simple and highly mobile hexapod robot, Int. J. Rob. Res. 20 (7) (2001) 616–631, https://doi.org/10.1177/02783640122067570.

[6] Y. Wang, M. Ramezani, M. Fallon, Actively mapping industrial structures with information gain-based planning on a quadruped robot, in: 2020 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2020, pp. 8609–8615, https://doi.org/10.1109/ICRA40945.2020.9197153.

[7] F. Kikuchi, Y. Ota, S. Hirose, Basic performance experiments for jumping quadruped, in: Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003), IEEE, 2003, pp. 3378–3383, https://doi.org/10.1109/IROS.2003.1249678.

[8] I D Miller, F. Cladera, A. Cowley, S S Shivakumar, E S Lee, L. Jarin-Lipschitz, A. Bhat, N. Rodrigues, A. Zhou, A. Cohen, Mine tunnel exploration using multiple quadrupedal robots, IEEE Robot. Autom. Lett. 5 (2) (2020) 2840–2847, https://doi.org/10.1109/LRA.2020.2972872.

[9] H-W Park, P M Wensing, S. Kim, High-speed bounding with the MIT Cheetah 2: control design and experiments, Int. J. Rob. Res. 36 (2) (2017) 167–192, https://doi.org/10.1177/0278364917694244.

[10] H W Park, P M Wensing, S. Kim, Online planning for autonomous running jumps over obstacles in high-speed quadrupeds, in: 2015 Robotics: Science and Systems Conference, RSS 2015, MIT Press Journals, 2015. http://hdl.handle.net/1721.1/97236.

[11] Q. Nguyen, M J Powell, B. Katz, J. Di Carlo, S. Kim, Optimized jumping on the mit cheetah 3 robot, in: 2019 International Conference on Robotics and Automation (ICRA), IEEE, 2019, pp. 7448–7454, https://doi.org/10.1109/ICRA.2019.8794449.

[12] H-W Park, P M Wensing, S. Kim, Jumping over obstacles with MIT Cheetah 2, Rob. Auton. Syst. (2021) 136103703, https://doi.org/10.1016/j.robot.2020.103703.

[13] F. Ehrlich-Sommer, F. Hoenigsberger, C. Gollob, A. Nothdurft, K. Stampfer, A. Holzinger, Sensors for digital transformation in smart forestry, Sensors 24 (3) (2024) 798, https://doi.org/10.3390/s24030798.

[14] L. Han, Q. Zhu, J. Sheng, C. Zhang, T. Li, Y. Zhang, H. Zhang, Y. Liu, C. Zhou, R. Zhao, Lifelike agility and play in quadrupedal robots using reinforcement learning and generative pre-trained models, Nat. Mach. Intell. (2024) 1–12, https://doi.org/10.1038/s42256-024-00861-3.

[15] A. Gupta, S. Savarese, S. Ganguli, L. Fei-Fei, Embodied intelligence via learning and evolution, Nat. Commun. 12 (1) (2021) 5721, https://doi.org/10.1038/s41467-021-25874-z.

[16] L. Frering, G. Steinbauer-Wagner, A. Holzinger, Integrating Belief-Desire-Intention agents with large language models for reliable human–robot interaction and explainable Artificial Intelligence, Eng. Appl. Artif. Intell. (2025) 141109771, https://doi.org/10.1016/j.engappai.2024.109771.

[17] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, V. Vanhoucke, Sim-to-real: learning agile locomotion for quadruped robots, Rob.: Sci. Syst. (2018), https://doi.org/10.15607/rss.2018.xiv.010.

[18] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, Learning quadrupedal locomotion over challenging terrain, Sci. Robot. 5 (47) (2020) eabc5986, https://doi.org/10.1126/scirobotics.abc5986.

[19] X B Peng, E. Coumans, T. Zhang, T-W Lee, J. Tan, S. Levine, Learning agile robotic locomotion skills by imitating animals, Rob.: Sci. Syst. (2020), https://doi.org/10.15607/RSS.2020.XVI.064.

[20] E. Johns, Coarse-to-fine imitation learning: Robot manipulation from a single demonstration, in: 2021 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2021, pp. 4613–4619, https://doi.org/10.1109/ICRA48506.2021.9560942.

[21] G. Bellegarda, C. Nguyen, Q. Nguyen, Robust quadruped jumping via deep reinforcement learning, Rob. Auton. Syst. (2024) 182104799, https://doi.org/10.1016/j.robot.2024.104799.

[22] Q. Liu, D. Xu, B. Yuan, Z. Mou, M. Wang, Distance-controllable long jump of quadruped robot based on parameter optimization using deep reinforcement learning, IEEe Access. (2023) 1198566–1198577, https://doi.org/10.1109/ACCESS.2023.3313637.

[23] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, Learning agile and dynamic motor skills for legged robots, Sci. Robot. 4 (26) (2019) eaau5872, https://doi.org/10.1126/scirobotics.aau5872.

[24] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, S. Levine, Learning to walk via deep reinforcement learning, Rob.: Sci. Syst. (2018), https://doi.org/10.15607/rss.2019.xv.011.

[25] Heess N, Tb D, Sriram S, Lemmon J, Merel J, Wayne G, Tassa Y, Erez T, Wang Z, Eslami S. Emergence of locomotion behaviours in rich environments. arXiv preprint, (2017). https://doi.org/10.48550/arXiv.1707.02286.

[26] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, Learning robust perceptive locomotion for quadrupedal robots in the wild, Sci. Robot. 7 (62) (2022) eabk2822, https://doi.org/10.1126/scirobotics.abk2822.

[27] D. Silver, A. Huang, C J Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M.J.N. Lanctot, Mastering the game of Go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489, https://doi.org/10.1038/nature16961.

[28] A. Reske, J. Carius, Y. Ma, F. Farshidian, M. Hutter, Imitation learning from mpc for quadrupedal multi-gait control, in: 2021 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2021, pp. 5014–5020, https://doi.org/10.1109/ICRA48506.2021.9561444.

[29] Q. Yao, J. Wang, S. Yang, C. Wang, H. Zhang, Q. Zhang, D. Wang, Imitation and adaptation based on consistency: A quadruped robot imitates animals from videos using deep reinforcement learning, in: 2022 IEEE International Conference on Robotics and Biomimetics (ROBIO), IEEE, 2022, pp. 1414–1419, https://doi.org/10.1109/ROBIO55434.2022.10011737.

[30] T. Li, Y. Zhang, C. Zhang, Q. Zhu, J. Sheng, W. Chi, C. Zhou, L. Han, Learning terrain-adaptive locomotion with agile behaviors by imitating animals, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2023, pp. 339–345, https://doi.org/10.1109/IROS55552.2023.10342271.

[31] L-M Schraick, F. Ehrlich-Sommer, K. Stampfer, O. Meixner, A. Holzinger, Usability in human-robot collaborative workspaces, Univers. Access. Inf. Soc. (2024) 1–14, https://doi.org/10.1007/s10209-024-01163-6.

[32] X B Peng, P. Abbeel, S. Levine, M. Van de Panne, Deepmimic: example-guided deep reinforcement learning of physics-based character skills, ACM Trans. Graph. 37 (4) (2018) 1–14, https://doi.org/10.1145/3197517.3201311.

[33] Peng X B, Kanazawa A, Malik J, Abbeel P, Levine S. Sfv: Reinforcement learning of physical skills from videos. 37(6), 1-14 (2018). https://doi.org/10.1145/3272127.3275014.

[34] Liu L, Hodgins J. Learning basketball dribbling skills using trajectory optimization and deep reinforcement learning. 37(4), 1-14 (2018). https://doi.org/10.1145/3197517.3201315.

[35] J. Ho, S. Ermon, Generative adversarial imitation learning, Adv. Neural Inf. Process. Syst. (2016) 29, https://doi.org/10.48550/arXiv.1606.03476.

[36] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, Commun. ACM 63 (11) (2020) 139–144, https://doi.org/10.1145/3422622.

[37] X B Peng, Z. Ma, P. Abbeel, S. Levine, A. Kanazawa, Amp: Adversarial motion priors for stylized physics-based character control, ACM Trans. Graph. 40 (4) (2021) 1–20, https://doi.org/10.1145/3450626.3459670.

[38] A. Escontrela, X B Peng, W. Yu, T. Zhang, A. Iscen, K. Goldberg, P. Abbeel, Adversarial motion priors make good substitutes for complex reward functions, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2022, pp. 25–32, https://doi.org/10.1109/IROS47612.2022.9981973.

[39] A E Maher, G. Burin, P G Cox, T W Maddox, S C Maidment, N. Cooper, E R Schachner, K T Bates, Body size, shape and ecology in tetrapods, Nat. Commun. 13 (1) (2022) 4340, https://doi.org/10.1038/s41467-022-32028-2.

[40] E R Schachner, R L Cieri, J P Butler, C. Farmer, Unidirectional pulmonary airflow patterns in the savannah monitor lizard, Nature 506 (7488) (2014) 367–370, https://doi.org/10.1038/nature12871.

[41] H. Zhang, S. Starke, T. Komura, J. Saito, Mode-adaptive neural networks for quadruped motion control, ACM Trans. Graph. 37 (4) (2018) 1–11, https://doi.org/10.1145/3197517.3201366.

[42] K A Daniels, J. Burn, A simple model predicts energetically optimised jumping in dogs, J. Exp. Biol. 221 (9) (2018) jeb167379, https://doi.org/10.1242/jeb.167379.

[43] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2017, pp. 23–30, https://doi.org/10.1109/IROS.2017.8202133.